SOFTWARE ENGINEERING MINI-PROJECT

CHAT APPLICATION

Real-Time Chat Application Using WebSockets

CODE AND BRANCH COVERAGE REPORT

PROJECT DETAILS

Course: Software Engineering

Semester: 5th Semester

Section: K

Branch: Computer Science and Engineering (CSE)

University: PES University

Academic Year: 2026–2027

TEAM MEMBERS

Sl. No.	SRN	Name
1	PES2UG24CS625	Amoghaa R
2	PES2UG24CS664	Vaibhav R
3	PES2UG25CS817	P S Harshavardhan
4	PES2UG24CS648	Pramath Ganesh Hegde

CODE COVERAGE REPORT

Real-Time Chat Application Using Socket Programming

1. Introduction

The Code Coverage Report presents the results of measuring how much of the application's source code was executed during automated testing.

Code coverage helps identify the portions of the application that have been exercised by test cases and highlights areas that may require additional testing.

The coverage analysis was performed using Pytest and pytest-cov on the main application module.

2. Objectives

The main objectives of code coverage analysis were:

- Measure the percentage of source code executed during testing.
- Identify statements that were not executed.
- Evaluate conditional and branching logic coverage.
- Identify areas that may require additional test cases.
- Verify the coverage result against the project's specified 80% target.

3. Testing Tools and Environment

Component	Details
Operating System	Windows
Programming Language	Python 3.14.2
Testing Framework	Pytest 9.2.0
Coverage Plugin	pytest-cov 7.1.0
Coverage Target	app.py
Branch Coverage	Enabled
Test Cases Executed	141

4. Coverage Measurement Methodology

Code coverage was measured using pytest-cov with branch coverage enabled.

The following command was used:

python -m pytest tests --cov=app --cov-branch --cov-report=term-missing --cov-report=html:tests/results/coverage_html --cov-report=xml:tests/results/coverage.xml --cov-report=json:tests/results/coverage.json

The command generated three coverage reports:

- HTML report for visual inspection of executed and missed code.
- XML report for structured coverage information.
- JSON report for numerical coverage data.

5. Coverage Results

The coverage execution completed with the following results:

Metric	Result
Total tests executed	141
Passed tests	141
Failed tests	0
Statements measured	986
Statements missed	158
Statements executed	828
Total branches	314
Partial branches	72
Combined coverage	82%

The measured application module achieved 82% combined coverage, exceeding the project's specified minimum target of 80%.

6. Coverage Analysis

The coverage results indicate that most of the measured application code was exercised during automated testing.

A total of 828 out of 986 statements were executed. The remaining 158 statements were not executed during the test run.

Branch coverage was also enabled to examine conditional paths in the application. The report identified 314 branches, with 72 partial branches.

The missed statements and partially covered branches indicate areas where additional test cases could improve coverage, particularly for exceptional conditions, alternative execution paths, and less frequently used operations.

7. Generated Coverage Reports

The following reports were generated during coverage execution:

Report	File Location
HTML Coverage Report	tests/results/coverage_html
XML Coverage Report	tests/results/coverage.xml
JSON Coverage Report	tests/results/coverage.json

The HTML report provides a detailed visual representation of covered and missed statements. The XML and JSON reports provide structured information for further analysis.

8. Conclusion

The code coverage analysis was successfully completed using Pytest and pytest-cov.

The measured application module achieved 82% combined coverage, exceeding the specified minimum target of 80%. All 141 automated tests passed during the coverage execution.

The analysis also identified statements and conditional branches that were not fully exercised, providing useful information for future testing improvements.

Overall, the coverage results demonstrate that the tested application module achieved the required coverage threshold.

BRANCH COVERAGE REPORT

Real-Time Chat Application Using Socket Programming

1. Introduction

This report presents the branch coverage analysis performed on the Real-Time Chat Application. Branch coverage measures whether the different possible execution paths of conditional statements are exercised during automated testing.

It helps identify conditional paths that have not been fully tested and provides insight into the completeness of the test suite.

2. Objectives

The main objectives of branch coverage analysis were:

- Measure the number of conditional branches executed during testing.
- Identify covered and missing branches.
- Evaluate the effectiveness of automated test cases in exercising different execution paths.
- Identify areas requiring additional test cases.
- Assess the application's conditional logic coverage.

3. Testing Environment

Component	Details
Operating System	Windows
Programming Language	Python 3.14.2
Testing Framework	Pytest 9.2.0
Coverage Plugin	pytest-cov 7.1.0
Coverage Target	app.py
Branch Coverage	Enabled
Total Test Cases	141

4. Branch Coverage Methodology

Branch coverage was measured using pytest-cov with the branch coverage option enabled.

The following command was executed:

python -m pytest tests --cov=app --cov-branch --cov-report=term-missing --cov-report=html:tests/results/coverage_html --cov-report=xml:tests/results/coverage.xml --cov-report=json:tests/results/coverage.json

The generated JSON coverage report was used to obtain the exact number of covered branches, missing branches, and branch coverage percentage.

5. Branch Coverage Results

The analysis produced the following results:

Metric	Result
Total branches	314
Covered branches	238
Missing branches	76
Partially covered branches	72
Branch coverage percentage	75.80%

The branch coverage percentage was calculated using:

Branch Coverage = (Covered Branches / Total Branches) × 100

= (238 / 314) × 100

= **75.80%**

6. Analysis of Results

The analysis shows that 238 out of 314 identified branches were covered during automated testing, resulting in 75.80% branch coverage.

A total of 76 branches were not covered, while 72 branches were identified as partially covered.

Partially covered branches indicate that only some possible execution paths of the corresponding conditional statements were exercised.

These results suggest that the existing test suite successfully exercises a majority of the application's conditional logic, while additional test cases could improve coverage of alternative conditions, error-handling paths, and less frequently executed scenarios.

7. Relationship with Code Coverage

Branch coverage and statement coverage measure different aspects of software testing.

Statement coverage measures whether individual executable statements are executed, whereas branch coverage measures whether alternative paths of conditional logic are exercised.

The application achieved 82% combined coverage and 75.80% branch coverage in the recorded coverage execution.

8. Conclusion

Branch coverage analysis was successfully performed using pytest-cov with branch tracking enabled.

The application achieved 75.80% branch coverage, with 238 out of 314 branches covered during automated testing.

The analysis also identified 76 missing branches and 72 partially covered branches, highlighting opportunities for improving test coverage.

Overall, the branch coverage report provides a clear assessment of the application's conditional execution paths and identifies areas for future testing improvements.